

Production GenAI Systems

RAG • Fine-Tuning • Evals • Observability • Caching • Feedback Loops • Guardrails

Introduction

Why a prototype:

Prompt → LLM → Response

becomes a production system involving retrieval, evaluation, monitoring, optimization, feedback, and safety.

1. RAG — Giving the Model the Right Knowledge

What is RAG?

Explain the basic idea and why LLMs need external knowledge.

How RAG works

Documents



Parsing



Chunking



Embeddings



Vector DB

User Query



Retrieval



Reranking



Context



LLM



Answer

Important concepts

Chunking — how documents are divided and why chunk size matters.

Embeddings — converting semantic meaning into vectors.

Vector databases — storing and retrieving relevant chunks.

Hybrid search — combining semantic and keyword search.

Reranking — improving the relevance of retrieved results.

Metadata filtering — restricting retrieval by things such as document type, geography, department, or version.

Common RAG failures

- Correct document not retrieved
- Wrong document ranked first
- Stale documents
- Poor chunking
- Too much context
- Hallucination despite correct context

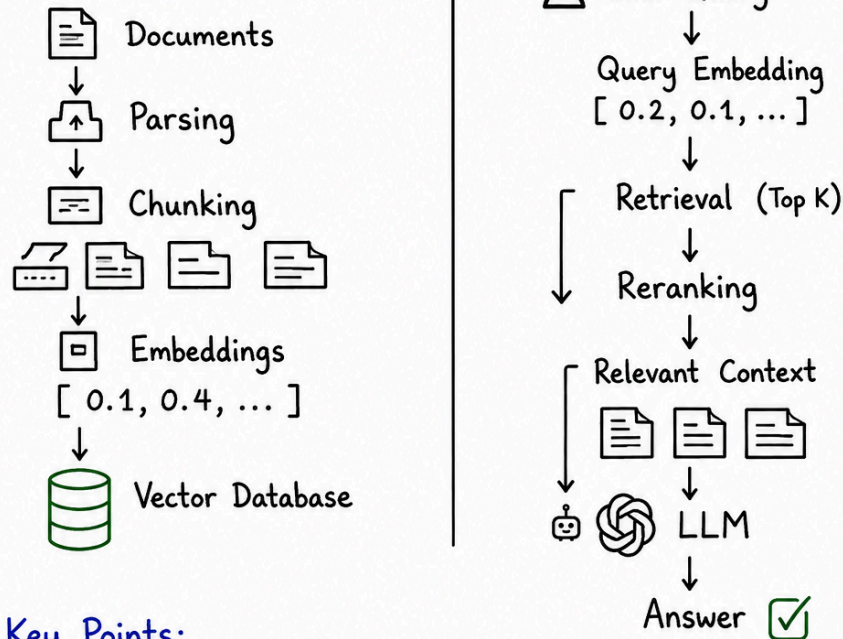
Key takeaway

RAG is primarily a knowledge problem, not a model-training problem.

1. RAG (Retrieval-Augmented Generation)

Goal: Give the LLM the right knowledge at the time of answering.

How it works:



Key Points:

- Good retrieval = good context = good answer
- Chunk size, overlap, metadata, embeddings
- Hybrid search + reranking improves quality
- Keep docs updated to avoid stale context

Common Failures:

- Wrong/irrelevant docs retrieved
- Poor chunking
- Too much context → confuse LLM
- Hallucination even with correct context

2. Fine-Tuning — Teaching the Model How to Behave

What is fine-tuning?

Explain how a pretrained model can be further trained using task-specific examples.

When to use it

Examples:

- Classification
- Specific output formats
- Consistent style
- Domain-specific behavior
- Specialized tasks

RAG vs Fine-Tuning

	RAG	Fine-Tuning
Changes	Context/knowledge	Model behavior
Knowledge freshness	Easy	Difficult
Private knowledge	Good	Expensive to maintain
Style	Limited	Strong
Classification	Possible	Often useful

LoRA / PEFT

Briefly explain parameter-efficient fine-tuning and why it reduces training cost.

Common mistakes

The biggest one:

Don't fine-tune a model simply because it doesn't know frequently changing information.

2. Fine-Tuning

Goal: Teach the model how to behave.
Not just what to know.

When to use:

- ✓ Consistent output format
- ✓ Specific style or tone
- ✓ Classification / Extraction
- ✓ Domain or task specific behavior

RAG vs Fine-Tuning

Aspect	RAG	Fine-Tuning
Changes	Context / Knowledge	Model Behavior
Knowledge Freshness	Easy (just update docs)	Hard (retrain)
Private / Internal Knowledge	Good	Expensive to maintain
Style / Format	Limited	Strong
Classification	Possible	Often useful
Cost (Long term)	Lower	Higher

LoRA / PEFT:

Fine-tune small additional parameters (adapters)
keeps most weights frozen → lower cost, faster.

Common Mistake:

Fine-tuning a model just because it doesn't know
frequently changing information.
Use RAG instead.

3. Evals — Testing AI Systems

This should be the **deepest section**.

Start with the problem:

Traditional software has unit tests. AI systems need evaluations because their outputs can be variable, subjective, and multi-step.

Use the terminology from the Anthropic document:

Task



Trial



Transcript



Grader



Outcome

Explain each one clearly.

Task

A defined test case with inputs and success criteria.

Trial

One execution of that task.

Transcript

What happened during execution.

Outcome

What actually happened to the system/environment.

Grader

How we determine whether the result was successful.

Evaluation Harness

The infrastructure that runs the task, captures the transcript, applies graders, and produces results.

Evaluation Suite

A collection of tasks testing a particular capability or behavior.

These definitions are directly aligned with the Anthropic source.

Types of Graders

Code-based

Use deterministic checks whenever possible.

Examples:

- Exact match
- Regex
- Database state
- API result
- Schema validation

Pros: fast, cheap, reproducible.

Cons: can be too rigid for subjective tasks.

Model-based

Use an LLM as a judge.

Examples:

- Rubric scoring
- Natural-language assertions
- Pairwise comparison
- Reference-based evaluation

Pros: flexible.

Cons: can be inconsistent and biased.

Human

Useful for:

- Expert judgment
- Difficult subjective cases
- Validating model-based graders

But expensive and difficult to scale.

Anthropic explicitly frames these as complementary rather than mutually exclusive approaches.

Capability vs Regression Evals

Capability

What can the system do?

Use challenging tasks where improvement is still possible.

Regression

Did something that worked before break?

These should ideally have very high pass rates.

Capability Evals

↓

Improve system

↓

Regression Evals

↓

Protect existing behavior

Anthropic recommends maintaining both.

Non-Determinism

LLMs may behave differently on different trials.

Explain:

pass@k

At least one of k attempts succeeds.

pass^k

All k attempts succeed.

Use a simple example:

Success probability = 80%

pass@5

→ probability of at least one success

pass^5

→ probability all five succeed

This distinction is important because **ability to eventually solve a task** is different from **consistently solving it**.

Designing a Good Eval Suite

Start small.

Anthropic recommends beginning with roughly **20–50 realistic tasks**, particularly tasks derived from real failures.

Good tasks should be:

- Clear
- Solvable
- Reproducible
- Representative
- Easy to grade

Also test both:

Should do X

Should NOT do X

Don't accidentally reward over-triggering.

And whenever possible:

Evaluate the outcome, not an unnecessarily rigid sequence of steps.

Evals in CI/CD

Code / Prompt Change



Run Eval Suite



Compare with Baseline



Regression?



Yes



No



Investigate



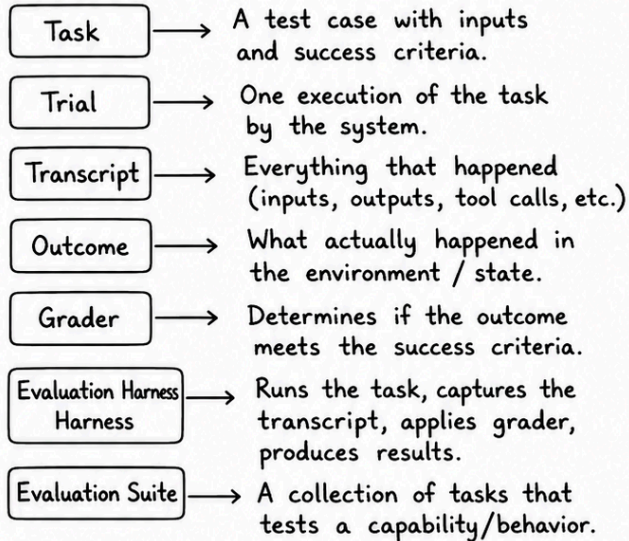
Deploy

This turns AI quality into an engineering process rather than subjective judgment.

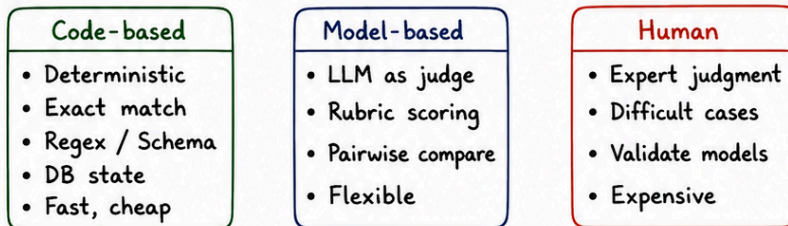
3. Evals (Evaluations)

Goal: Know if the system works, and make it better over time.

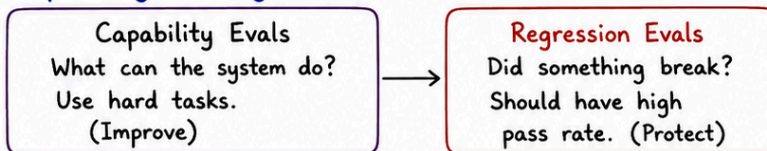
Core Concepts (Anthropic Framework):



Types of Graders:



Capability vs Regression



Non-Determinism:

pass@k: at least one of k trials succeeds.
pass^k: all k trials succeed.

Example:

Success prob = 80%
 $\text{pass}@5 = 1 - (0.2)^5 = 99.68\%$
 $\text{pass}^5 = (0.8)^5 = 32.77\%$

4. Observability — Understanding What Happened

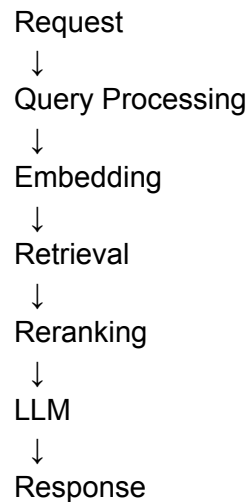
Evals answer:

Did it work?

Observability answers:

Why did it work or fail?

A production trace might look like:



For each stage, capture relevant information.

Important signals

Latency

- p50
- p95
- p99

Cost

- Input tokens
- Output tokens
- Cost/request

Quality

- Eval scores
- Feedback
- Task completion

RAG

- Retrieved documents
- Similarity scores
- Context size

Versioning

- Model version
- Prompt version
- Retrieval version

Why this matters

Suppose answer quality drops from 90% to 75%.

Observability should help determine whether:

Retrieval changed?

Prompt changed?

Model changed?

Data became stale?

Latency increased?

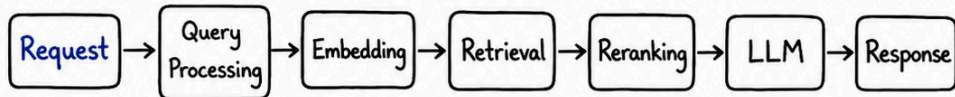
External API failed?

The goal is to make the AI system **debuggable rather than mysterious**.

4. Observability

Goal: Understand what happened in the system.
Make it debuggable.

Trace (Example):



What to observe:



Performance

- Latency (p50/p95/p99)
- Throughput



Cost

- Input tokens
- Output tokens
- Cost / request



Quality

- Eval score
- User feedback
- Task success rate



RAG Signals

- Retrieved docs
- Similarity score
- Context size

Also track:

- Errors & exceptions
- Model version
- Prompt version
- Retrieval / Index version
- Tool / API calls & results

Why it matters?

When quality drops, observability helps answer:

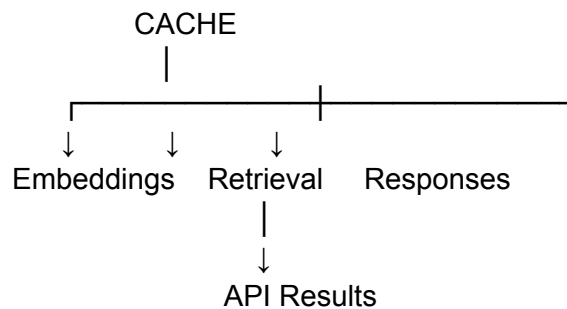
- Retrieval changed?
- Prompt changed?
- Model changed?
- Data became stale?
- External API failed?

5. Caching — Making AI Faster and Cheaper

LLM calls are expensive.

Caching avoids repeating work.

Different caching layers



Exact caching

Same request → reuse response.

Semantic caching

Different wording, same intent → potentially reuse response.

Embedding caching

Don't regenerate embeddings for unchanged documents.

Retrieval caching

Reuse retrieved documents for repeated queries.

API caching

Reuse external results when freshness allows it.

The hard part: invalidation

The key question isn't:

"Can we cache this?"

It's:

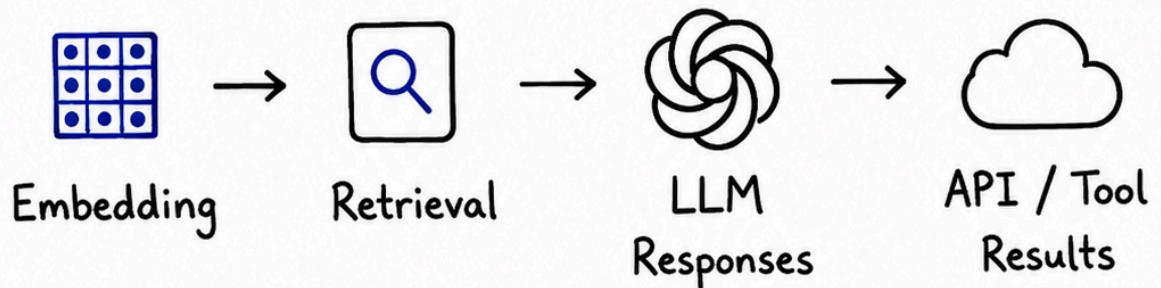
"When does the cached value become invalid?"

Caching can reduce latency and cost dramatically, but stale or incorrectly scoped caches can create correctness and even security problems.

5. Caching

Goal: Make the system faster and cheaper.
Avoid repeating expensive work.

Caching Layers:



Types:

- **Exact Caching:** same request → same response
- **Semantic Caching:** different wording, same intent
- **Embedding Caching:** don't recompute embeddings
- **Retrieval Caching:** reuse retrieved docs
- **API Caching:** reuse external API results

The Hard Part: Invalidation

Not the question:

"Can we cache this?"



The real question:

"When does it become invalid?"

Freshness > Speed



6. User Feedback Loops — Learning From Production

Offline evals cannot predict every real-world failure.

Users provide signals:



Corrections

Retries

Escalations

Abandoned workflows

But feedback isn't automatically ground truth.

A thumbs-down could mean:

- Wrong answer
- Wrong retrieval
- Missing information
- Bad UX
- User misunderstanding

So the important loop is:

User Feedback



Failure Analysis



Root Cause



Fix



New Eval



Regression Protection

Anthropic describes production feedback as particularly valuable for discovering unexpected problems, while also noting that feedback can be sparse and biased toward severe failures.

The best systems don't just collect feedback.

They **convert important feedback into permanent engineering knowledge.**

6. User Feedback Loops

Goal: Learn from real users and improve continuously.

User Signals:



Corrections



Retries



Escalations



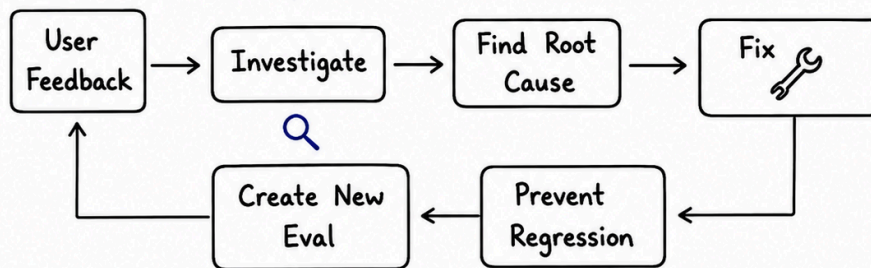
Abandoned
Workflows

Feedback $\neq$ Ground Truth

Thumbs down could mean:

- Wrong answer
- Wrong retrieval
- Missing information
- Bad UX
- User misunderstood
- Others...

The Feedback Loop:



Key Point:

Convert important feedback into permanent engineering knowledge (evals).

7. Guardrails — Controlling AI Behavior

Guardrails provide boundaries around the model.

Think of three layers:

User



Input Guardrails



LLM

↓
Output Guardrails
↓
Application

Input guardrails

Examples:

- Prompt injection detection
- PII detection
- Input validation
- Authentication
- Authorization

Output guardrails

Examples:

- Schema validation
- Safety checks
- PII detection
- Policy validation

Business rules

Important business logic shouldn't depend solely on the LLM.

For example:

LLM:

"Refund customer ₹50,000"

↓

Business Rule:

Maximum automated refund = ₹10,000

↓

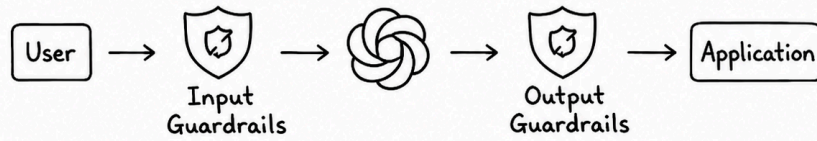
Block / Human approval

The model can **propose**, while deterministic application logic **enforces**.

7. Guardrails

Goal: Control what the system is allowed to do.
Protect users, data, and business.

Layers:



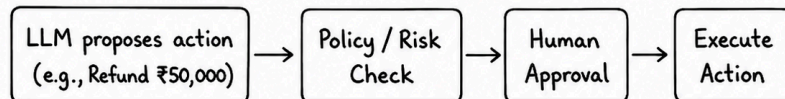
Input Guardrails (Examples)

- Prompt injection detection
- PII detection
- Input validation
- Authentication
- Authorization

Output Guardrails (Examples)

- Schema validation
- PII / Sensitive data check
- Content safety / policies
- Toxicity / Harm detection

Business Rules & Humans-in-the-Loop



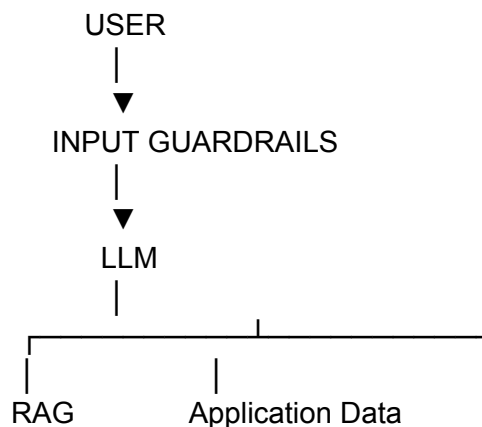
Key Principle:

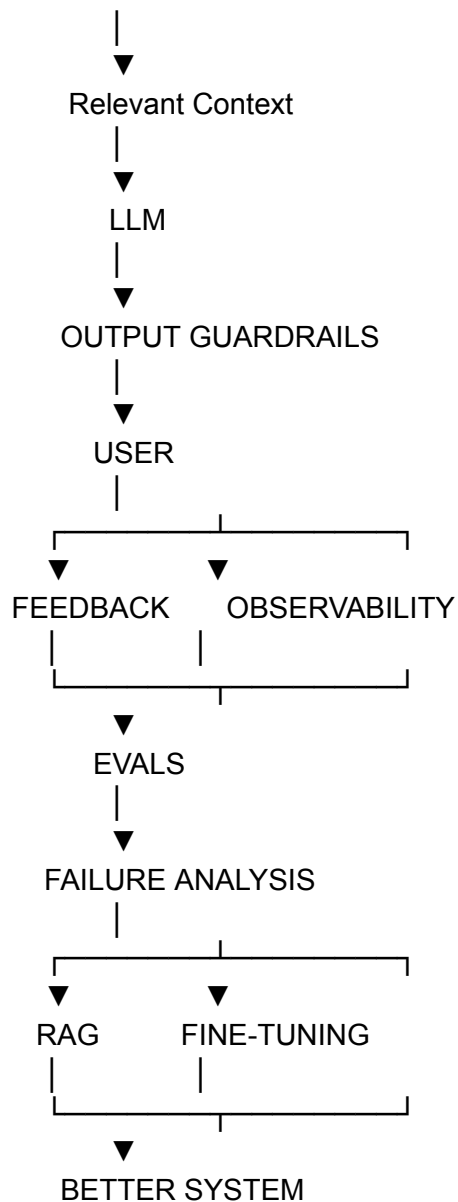
Model can propose.
Application decides.



8. How Everything Fits Together

This is the most important section.

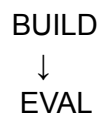


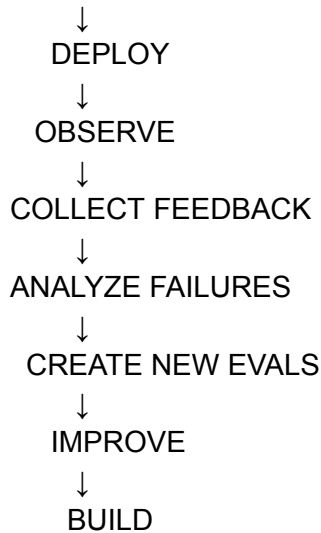


And **caching** operates across the expensive components.

9. The Production Feedback Loop

The entire system eventually becomes:





This is the real difference between an **LLM demo** and a **production AI system**.

The model is only one component.

The engineering system around the model determines whether the application is:

reliable, measurable, safe, scalable, and continuously improving.

10. The One-Minute Summary

If someone asks you to explain the whole stack:

RAG gives the model the right information.

Fine-tuning teaches it specialized behavior.

Evals tell us whether it works.

Observability tells us what happened.

Caching makes it faster and cheaper.

User feedback tells us what we're missing.

Guardrails keep the system within acceptable boundaries.

And the most important relationship is:

Production failures → feedback → evals → improvements → fewer future failures.

That is the core of building reliable GenAI systems.